ZENIKA · ATELIER IA · DÉVELOPPEURS · 3H30

Context Engineering

Reprendre le contrôle des LLM et des agents de code

Un bon résultat vient moins d'un prompt magique que d'une **context window bien composée**.

Agenda

1. Ouverture — 15 min
2. Sentir le non-déterminisme — 35 min
3. Formaliser : le trampoline — 45 min
4. Pause + débrief — 15 min
5. Composer la context window — 50 min
6. Extraire le contexte utile — 35 min
7. Clôture — 15 min

© ZENIKA 2026 | All rights reserved

Une session subie

Une scène familière

Ajoute la pagination sur l'endpoint /bookings.

[pagination offset, nouveau helper maison]

Non, on utilise des curseurs.

[curseurs, mais casse le contrat OpenAPI]

Tu as oublié les tests…

[tests qui mockent tout]

Recommence, plus simple.

Ce qu'il fallait donner dès le départ

- « ajoute la pagination » → un objectif et un critère de fin
- « on utilise des curseurs » → les conventions du projet
- « le contrat OpenAPI » → les invariants
- « les tests » → les commandes de vérification

i Chaque relance révèle une information qu'on avait en tête. On ne converse pas jusqu'à ce que le modèle devine : on **construit son entrée**.

© ZENIKA 2026 | All rights reserved

Ce qu’on refuse, ce qu’on privilégie

TokenMaxxing

DEMANDE
Un prompt vague, on verra bien

CONTEXTE
Un dump de fichiers « au cas où »

BOUCLE
Un tool en boucle, des erreurs empilées

DÉRIVE
Des corrections nerveuses par-dessus

Approche ingénieur

OBJECTIF
Un objectif clair et un critère de fin

CONTEXTE CHOISI
Les fichiers et commandes utiles à la tâche

VALIDATION
Une sortie intermédiaire vérifiée

REWIND
On corrige l’entrée et on relance

i Le résultat du TokenMaxxing peut impressionner. En équipe, il coûte cher et **fragilise la base de code.**

© ZENIKA 2026 | All rights reserved

Démo + activité 1 · 35 min

Sentir le non-déterminisme

Relancer le même prompt pour mesurer la largeur de la distribution.



© ZENIKA 2026 | All rights reserved

Quatre notions pour piloter un LLM

1



Modèle probabiliste

Le LLM produit le token suivant le plus plausible, tiré avec une part de hasard.

RÉFLEXE

une trajectoire plausible, pas la vérité

2



Attention

Chaque token déjà présent pèse sur la probabilité du suivant.

RÉFLEXE

chaque mot de l'entrée compte

3



Autorégression

La sortie du modèle redevient son entrée suivante.

RÉFLEXE

une erreur générée guide la suite

4



Context window

Tout ce que le modèle voit pendant une génération. Limitée, coûteuse.

RÉFLEXE

plus de bruit, moins d'attention

i Piloter un LLM, c'est **choisir ce qui entre** dans sa context window.

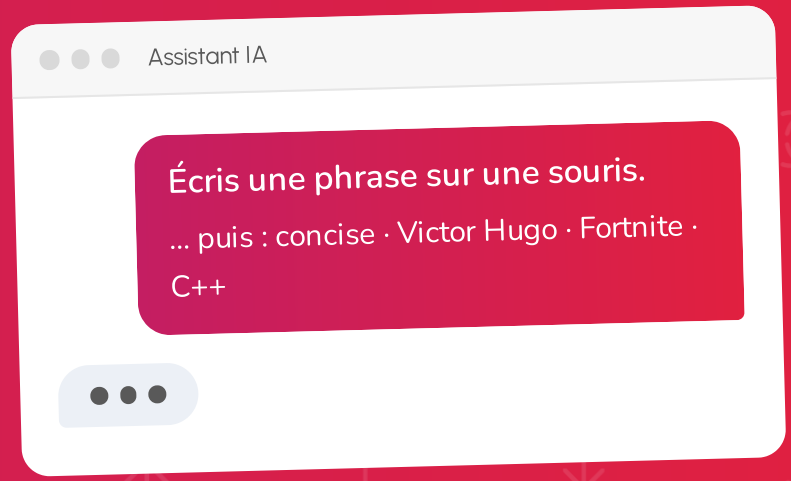
© ZENIKA 2026 | All rights reserved

▶ DÉMO

Une souris, plusieurs trajectoires

Un prompt, cinq relances, un mot intrus.

Le modèle ne « comprend » pas l'intrus : il le transforme en signal statistique.



© ZENIKA 2026 | All rights reserved

Activité — Regenerate

EXERCICE 10 min

Avec votre agent ou assistant

1. Écrivez une demande de code courte, volontairement **floue**.
2. Relancez-la **5 fois**. Notez ce qui reste stable.
3. Changez **3 mots** au maximum : langage, contrainte, format.
4. Ajoutez un **intrus** et comparez.

Exemple

Départ :

```
Écris une fonction qui valide un email.
```

Puis, avec trois mots de plus :

```
En TypeScript, sans regex, avec tests :
écris une fonction qui valide un email.
```

i Cherchez les mots qui **resserrent** la sortie et ceux qui la **contaminent**.

© ZENIKA 2026 | All rights reserved

Rewind plutôt qu'empiler



i Traitez une mauvaise sortie comme un **message du compilateur** : on corrige l'entrée, on relance.

© ZENIKA 2026 | All rights reserved

Démo + activité 2 · 45 min

Formaliser : le trampoline

Du langage naturel vers une structure intermédiaire, puis vers un artefact.



© ZENIKA 2026 | All rights reserved

Le trampoline autorégressif

1



Idée floue

Le besoin tel qu'on le formule en une phrase.

EXEMPLE

« Un browser d'animaux. »

2



Structure validée

Le modèle propose un YAML, un type, un schéma.
L'humain le relit et le corrige.

EXEMPLE

species, diet, habitat

3



Artefact

Généré à partir de la structure validée, pas de l'idée floue.

EXEMPLE

HTML single-file, CLI, doc

i La sortie validée du tour 1 devient le **contexte** du tour 2 : on construit étage par étage.

© ZENIKA 2026 | All rights reserved

▶ DÉMO

Formaliser avant de coder

D’abord la structure, ensuite seulement le code.

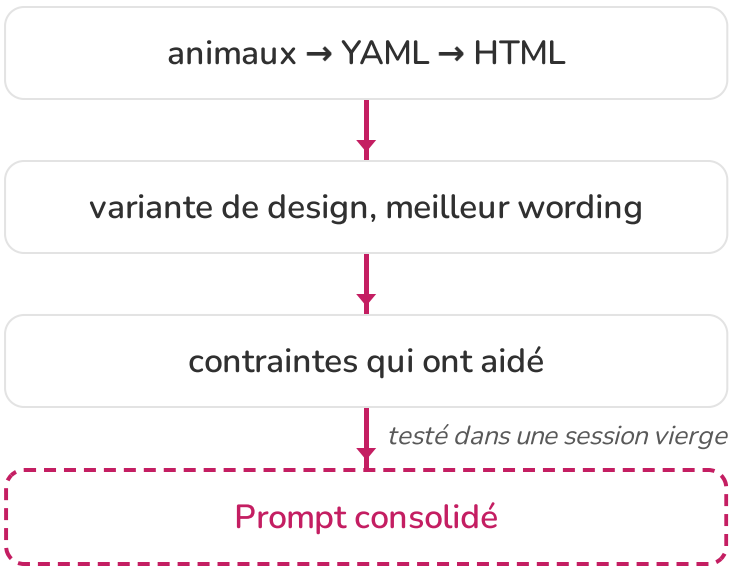
Le YAML est un contrat temporaire : l'humain valide l'abstraction avant la génération.



© ZENIKA 2026 | All rights reserved

Du prototype au prompt consolidé

Expériences locales



Le vrai livrable

```
À partir d'un domaine, crée une structure
YAML minimale et attends ma validation.
Puis génère une page HTML single-file :
filtres sur chaque champ, sans dépendance.
Exemple de structure attendue :
  items:
    - name: …
      category: …
```

- Testé sur deux domaines : animaux, puis pâtisserie.

i Un prompt qui marche sur deux domaines est un **actif** : instructions projet, skill d'équipe.

© ZENIKA 2026 | All rights reserved

Activité — Formaliser

EXERCICE 25 min

Avec votre agent ou assistant

1. Choisissez un domaine : métier ou ludique (inventaire, backlog, plantes, jeux).
2. Faites produire une structure intermédiaire, sans code.
3. Corrigez-la, puis faites générer l'artefact.
4. Consolidez le prompt et testez-le sur un second domaine, dans une session vierge.

La chaîne

```
idée
  ↓
YAML · types · JSON Schema · OpenAPI
  ↓
HTML single-file · CLI · mini-doc
  ↓
prompt consolidé, testé à vide
```

i **Débrief** : montrez le **prompt consolidé**, pas seulement le rendu.

© ZENIKA 2026 | All rights reserved

Pause + débrief · 15 min

On souffle

Qu’est-ce que la formalisation a réglé ? Qu’est-ce qu’elle n’a pas réglé ?



© ZENIKA 2026 | All rights reserved

Ce que le trampoline règle… et ne règle pas

Ce qu'il règle

AMBIGUÏTÉ
Les champs sont nommés et visibles

FORMAT
La sortie suit la structure

VARIANCE
Les relances convergent

Ce qu'il ne règle pas

OBJECTIF
Un mauvais objectif reste mauvais

ARCHITECTURE
Une mauvaise architecture reste mauvaise

SESSION
Une session agentique sale reste sale

On change d'échelle : on ne formalise plus une petite structure, on prépare la context window d'un **agent dans un vrai projet**.

© ZENIKA 2026 | All rights reserved

Démo + activité 3 · 50 min

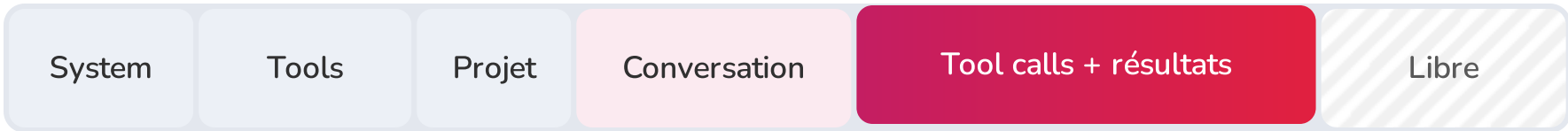
Composer la context window

Une fenêtre efficace est dense en signaux utiles et pauvre en bruit.



© ZENIKA 2026 | All rights reserved

La context window d’un agent de code



CONTEXT WINDOW D’UN AGENT DE CODE — TOUT CE QUE LE MODÈLE REÇOIT À CHAQUE APPEL

Tool calls + résultats

Fichiers lus, sorties de commandes, logs d’erreur, résultats de tests : le bloc qui grossit le plus vite.

C’est l’**entrée effective** du modèle : bien plus que votre dernier message.

Chaque tool exposé, chaque fichier lu, chaque sortie de commande **consomme du budget.**

© ZENIKA 2026 | All rights reserved

Le modèle n’a pas de mémoire

À chaque appel, le harness renvoie **toute la session** au modèle.



- Une erreur du tour 2 est **relue à chaque tour** suivant.
- Une session longue coûte **de plus en plus cher** à chaque appel.

> *i* Une **nouvelle session** repart de zéro : c’est un outil, pas un échec.

© ZENIKA 2026 | All rights reserved

Plus n’est pas mieux



Simple

Objectif, format, un exemple, la demande.

EFFET

Attention concentrée.



Agentique

Tool schemas, fichiers lus, sorties de commandes.

EFFET

Les résultats des tools dominent la demande.



Polluée

Objectif périmé, dump, API hallucinée, corrections.

EFFET

Des sorties crédibles mais fausses.



Préparée

Carte de la tâche, fichiers clés, commandes, invariants.

EFFET

Moins d’exploration à l’aveugle.

i La bonne question : **ce qui est dans la fenêtre aide-t-il la tâche du jour ?**

© ZENIKA 2026 | All rights reserved

Ce que le harness injecte



SESSION AGENTIQUE · LE HARNESS INJECTE BIEN PLUS QUE VOS MESSAGES

Sorties de commandes

Un build, un npm install ou une stack trace peuvent dominer la context window s'ils ne sont pas filtrés.

Un agent fait beaucoup plus qu'un chat… et remplit la fenêtre avec beaucoup plus que vos messages.

i La question n'est pas « l'agent est-il intelligent ? », mais « **ce qu'il a injecté aide-t-il la tâche ?** »

© ZENIKA 2026 | All rights reserved

Ouvrir la context window

1



Regarder

L'occupation du contexte, les tool calls, les fichiers lus : la plupart des harness les affichent.

RÉFLEXE

savoir ce qui est entré

2



Demander

« Quels fichiers as-tu lus ? Sur quelle hypothèse pars-tu ? »

RÉFLEXE

rendre visible le raisonnement

3



Élaguer

Retirer ce qui ne sert plus : MCP inutiles, vieux objectifs, gros logs.

RÉFLEXE

rewind ou session propre

i Si on ne sait pas ce qui est entré dans la fenêtre, on ne peut pas **expliquer la sortie**.

© ZENIKA 2026 | All rights reserved

Une carte pour la tâche du jour

Le principe

- Pas une carte générale du dépôt : une carte orientée tâche.
- Courte et relue : mieux que 3 000 lignes génériques.
- Les noms sont des ancres : un chemin, une commande, un invariant orientent l'attention vers tout un bloc de code.

Exemple

```
# Tâche : pagination par curseur sur /bookings
## Fichiers clés
- src/bookings/bookings.controller.ts
- src/shared/pagination/cursor.ts
## Commandes
- npm test -- bookings
- npm run lint
## Invariants
- ne pas casser le contrat OpenAPI v2
- curseur opaque, encodé en base64
## Pièges connus
- les tests e2e ont besoin de la base locale
```

i En la relisant, on découvre souvent les **zones floues du projet**… avant de lancer l'agent.

© ZENIKA 2026 | All rights reserved

Activité — Cartographier

EXERCICE 25 min

Sur votre dépôt

1. **Cibler** une tâche réelle : ajouter un endpoint, réduire une duplication, écrire des tests.
2. **Générer** une carte orientée tâche : fichiers clés, commandes, conventions, pièges.
3. **Relire et corriger** : vérifier les noms, retirer le bruit.
4. **Tester** : nouvelle session, carte chargée, même tâche.

Le test de vérité

```
session propre + tâche claire
          ↓
carte générée
          ↓
relecture humaine + corrections
          ↓
nouvelle session + carte chargée
          ↓
agent plus direct, moins d'exploration
```

i **Débrief** : qu'avez-vous corrigé dans la carte ? L'agent a-t-il moins exploré ?

© ZENIKA 2026 | All rights reserved

Démo + activité 4 · 35 min

Extraire le contexte utile

Une session ratée n’est pas un déchet. C’est du minerai.



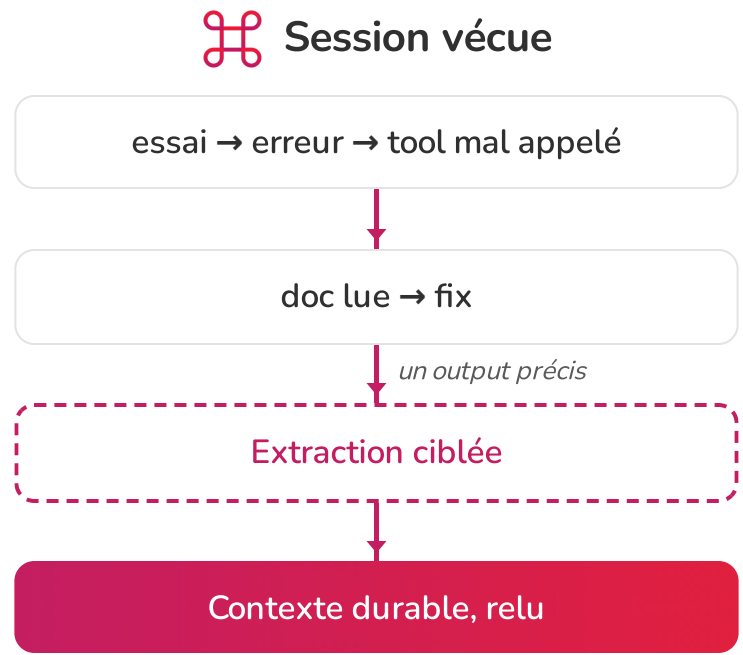
© ZENIKA 2026 | All rights reserved

Exploiter les traces

Une session contient

- ce qui a été **tenté**
- ce qui a **échoué**, et pourquoi
- ce qui **manquait** : doc, outil, convention
- ce qui a **fini par marcher**

C'est de l'**inférence déjà payée**.



> *i* Si l'agent galère avec une API, ce n'est pas toujours le prompt : c'est souvent **l'API, la doc ou l'outil** qui parle mal.

© ZENIKA 2026 | All rights reserved

Recettes d’extraction

« Résume cette session » est moins utile qu’une **extraction ciblée** :

Fichier	Contenu
`user-guide.md`	comment utiliser l’outil, commandes qui marchent, pièges
`troubleshooting.md`	erreur → cause → fix
`api-improvements.md`	signatures, noms, messages d’erreur, docs à corriger
`new-tool.md`	le script ou le tool qui aurait évité le frottement
`next-session.md`	ce qu’on recharge dans une session propre

> *i* Une extraction **non relue** n’est jamais une source d’autorité.

© ZENIKA 2026 | All rights reserved

Activité — Extraire

EXERCICE 20 min

Sur une session existante

1. **Identifier** les frottements : tool mal appelé, API mal comprise, doc manquante, erreur répétée.
2. **Extraire** un seul output ciblé : guide, troubleshooting, proposition d'API, outil.
3. **Relire** : élaguer, corriger, densifier.

Exemple

```
Extrais de cette session un troubleshooting :
pour chaque erreur rencontrée, une ligne
« erreur → cause → fix ».
Garde seulement ce qui a été vérifié.
Si une cause est supposée, écris-le.
N'invente rien.
```

i **Débrief** : qu'avez-vous dû corriger ? Où ce fichier va-t-il vivre ?

© ZENIKA 2026 | All rights reserved

Où ranger le contexte durable

1



Instructions projet

Conventions, commandes, pièges du dépôt. Versionnées, courtes.

> *AGENTS.md à la racine*

CHARGÉ
à chaque session

2



Carte de tâche

Fichiers clés, invariants, plan pour une tâche donnée.

> *docs/tasks/pagination.md*

CHARGÉ
pour cette tâche

3



Skill

Un savoir-faire emballé : consignes, exemples, scripts.

> *le prompt consolidé du module 2*

CHARGÉ
à la demande

i Plus c'est chargé souvent, plus c'est **court** : tout ce qui est chargé pèse dans **chaque** appel.

© ZENIKA 2026 | All rights reserved

Clôture · 15 min

Ce qu’on garde



© ZENIKA 2026 | All rights reserved

La boucle complète

1

Expérimenter

relancer, comparer : sentir la distribution (module 1)

2

Formaliser

une structure validée avant l'artefact (module 2)

REPARTIR D'UNE SESSION PROPRE

3

Cartographier

une carte dense pour la tâche du jour (module 3)

4

Extraire

transformer les traces en contexte durable (module 4)

i À chaque étape : **choisir ce qui entre** dans la fenêtre, **relire avec un œil humain.**

© ZENIKA 2026 | All rights reserved

5 réflexes pour lundi matin

TIPS

1. **Relancer** avant de croire qu'un prompt est robuste.
2. Donner **un exemple du format** plutôt qu'un long paragraphe.
3. **Rewind** dès que la session dérive, au lieu d'empiler les corrections.
4. Écrire une **carte de la tâche** avant de lancer un agent dans un gros projet.
5. **Extraire** les leçons d'une session avant de la jeter.

> *i* Moins de volume, plus de **signaux utiles** : moins de frustration, plus de contrôle.

© ZENIKA 2026 | All rights reserved

Les mots-clés à retenir

Modèle

LLM **token**

attention

autorégression

non-déterminisme

hallucination

Demande

prompt few-shot

rewind trampoline

prompt consolidé

Contexte

context window

context engineering

context rot

compaction session

invariant

Agent

agent de code

harness tools MCP

instructions projet

skill

© ZENIKA 2026 | All rights reserved

Questions

Qu’est-ce que vous changez dans votre prochaine session avec un agent ?

© ZENIKA 2026 | All rights reserved